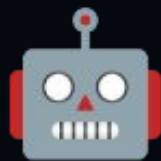


SEMINAR · 2026.07 · ORCHESTRATION

서브에이전트와 오케스트레이션



혼자가 아니라, *팀으로*.

에이전트 하나가 버거워지는 순간 — 팀을 꾸리는 법을 배웁니다.

[← 선수 세미나: 컨텍스트 — AI 에이전트의 작업 책상](#)



에이전트의 조직도

유능한 신입 한 명에게 전부 맡기면 - 방향키로 진행하세요 ▶

- 😊 1주차 — "네! 명세 전부 파악했습니다"

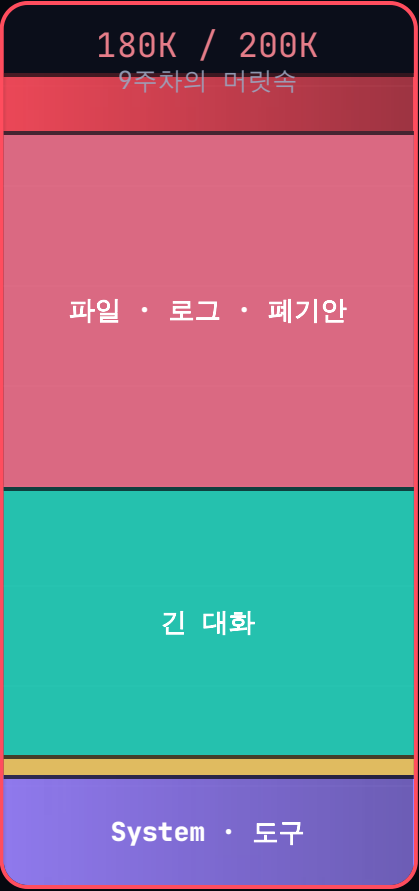
순항
- 📁 4주차 — 메모·회의록·시안이 머릿속에 산더미

과적
- 🌀 7주차 — 폐기한 v1 시안과 최신 v3가 뒤섞이기 시작

혼선
- 😓 9주차 — "저... 원래 목표가 뭐였죠?"

표류

에이전트 하나에 긴 작업을 쥐도 정확히 같은 일이 벌어집니다.
컨텍스트 창은 유한하고, 길어질수록 산으로 갑니다 — 왜 그런지는 [context 세미나](#)에서 다뤘습니다. 오늘은 "그래서, 어떻게".



한 명의 한계

사람 조직이 찾은 답 — 분업 · 위임 · 보고



분업

큰일을 독립된 조각으로 나눕니다. 조각마다 담당자 한 명.



위임

담당자에게는 회사의 모든 역사가 아니라 **임무 요약서**만 전달합니다.



보고

위로 올라가는 건 결과 요약뿐. 팀장은 실무자의 삽질 로그를 다 읽지 않습니다.

수천 년 동안 검증된 프로토콜입니다 —
AI 에이전트도 정확히 같은 답에 도달했습니다.

회사가 괜히 팀·파트·그룹으로 나뉜 게 아닙니다. 한 사람의 머리가 유한해서입니다.

오늘의 지도

여섯 걸음 — 원리에서 자동화까지

CH1 한 명의 한계

왜 팀인가 — 방금 봤습니다 ✓

CH2 서브에이전트의 원리

격리 · 위임장 쓰는 법 · 그리고 위임의 대가

CH3 패턴 4가지

역할 분담 · 병렬 fan-out · 파이프라인 · 심판 패턴

CH4 조율의 실전 도구

작업 목록 · 파일 공유 · worktree · 검증 게이트

CH5 사람이 없는 곳에서도

headless · CI · 스케줄 — 자는 동안 일하는 팀

CH6 함정과 판단 기준

언제 팀이고, 언제 그냥 혼자가 나은가

원리는 세 박자 — 위임 → 격리 → 보고



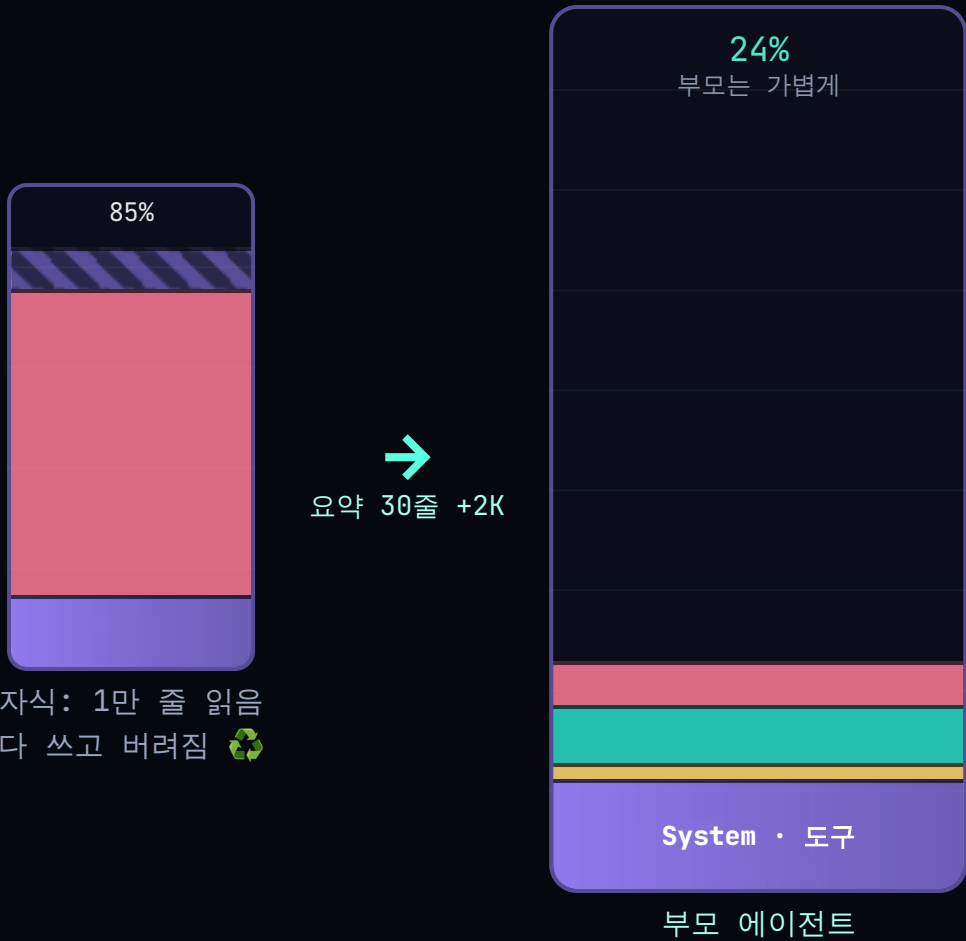
① 📧 부모는 임무를 요약해 위임합니다 — 대화 전체가 아니라 지시서 한 장

② 🧠 자식은 **깨끗한 자기 컨텍스트**에서 일합니다 — 부모의 긴 히스토리를 모른 채

③ 📋 돌아오는 건 요약뿐 — 자식이 읽은 1만 줄은 부모 상자에 실리지 않습니다

그래서 부모의 상자는 **계획과 결론만**으로 가볍게 유지됩니다.

핵심은 컨텍스트 격리입니다



자식이 읽은 1만 줄은 자식의 상자에서 소멸하고, 부모에게는 결론 30줄만 착륙합니다.

함수 분리와 같은 원리

지역 변수 = 자식의 탐색·삽질 반환값 = 요약 보고 호출자 = 부모

컨텍스트 세미나의 "서브에이전트 = 컨텍스트 격리" 한 장, 기억하시나요 — 오늘 세
미나 전체가 그 확장판입니다.

위임 프롬프트는 업무 지시서입니다

❌ 이렇게 위임하면
"auth 쪽 한번 봐줘"

자식은 부모의 대화를 모릅니다 —
무엇을? 왜? 어디까지?
게다가 중간에 되물을 수도 없습니다

✅ 업무 지시서

"src/auth의 토큰 갱신 로직을 조사해줘."

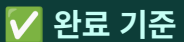
목표: 간헐적 로그아웃의 원인 후보 찾기

완료 기준: 파일:줄 위치와 근거 코드 2~3곳

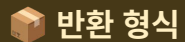
반환 형식: 표 - 위치 | 추정 원인 | 확신도"



목표



완료 기준



반환 형식

프롬프트 세미나의 3요소 프레임이 여기서 재등장합니다 — 위임에서는 특히 반환 형식이 생명입니다. 보고서 양식까지 정해주는 상사가 됩니다.

자식은 한 번 나가면 돌아올 때까지 질문하지 못합니다 — 지시서에 다 답으세요.

실전 — 말로 시키거나, 파일로 고용하거나

🗨️ 말로 시키기 — 이걸로 충분

"서브에이전트로 나눠서 조사해줘"
"독립적인 부분은 병렬로 진행해줘"
"리뷰는 별도 에이전트에게 맡겨줘"

이 한마디면 에이전트가 **알아서** 팀을 꾸립니다.

📁 커스텀 서브에이전트 — 파일로 채용 공고

```
.claude/agents/reviewer.md
---
name: reviewer
description: 코드 리뷰 전문.
  PR·diff 검토 시 사용
tools: Read, Grep
---
너는 깐깐한 시니어 리뷰어다. ...
```

이름·전문 분야·도구 권한을 정의해두면, 필요할 때마다 같은 전문가가 소환됩니다. tools 제한 = 읽기 전용 리뷰어처럼 권한 최소화.

다만, 위임은 공짜가 아닙니다



맥락 전달 비용

자식은 부모의 대화를 모릅니다 — 필요한 배경은 지시서에 다시 써서 들려 보내야 합니다



왕복 오버헤드

새 상자 준비, 같은 파일 다시 읽기, 보고 대기 — 작은 일에는 배보다 배꼽이 큼니다



요약 손실

돌아오는 건 요약뿐 — 부모는 세부 근거를 직접 본 적이 없습니다. 중요한 판단이면 근거를 반환 형식에 포함시키세요

위임은 투자입니다 — **격리의 이득이 오버헤드보다 클 때만.**

그래서 오케스트레이션의 절반은 "언제 위임하지 않을지"를 아는 것입니다 (CH6에서).

팀을 짜는 패턴 4가지

① 🧑‍🤝‍🧑 역할 분담

탐색 → 계획 → 구현 → 리뷰. 단계마다 다른 에이전트, 각자 깨끗한 눈.

읽을 건 많고 단계가 뚜렷할 때

② 🍴 병렬 fan-out

독립 작업 여러 개를 동시에. 벽시계 시간을 접습니다.

파일 10개 마이그레이션 · 다각도 조사

③ 🏗️ 파이프라인

항목별로 단계를 흘려보내기 — 앞 단계 전체를 기다리지 않고.

항목 많고 단계가 고정일 때

④ ⚖️ 심판 패널

같은 문제를 N번 독립 시도 → 채점·종합. 서로 반박도 시킵니다.

답 공간이 넓고 품질이 승부일 때

이제 한 장씩 — 각 패턴이 언제, 왜 이기는지 보겠습니다.

단계마다 깨끗한 눈을 새로 삽니다



탐색이 읽은 파일 수십 개는 탐색 상자에서 폐기 — 계획은 **정제된 보고서 위에서만** 세웁니다.

리뷰어는 구현자의 변명을 모릅니다. 본인 코드는 본인 눈에 늘 옳아 보이거든요 — 사람도, 에이전트도.

독립 작업은 동시에 — 벽시계 시간을 접습니다.

✗ 혼자 순차 – 파일 5개 × 10분

에이전트 1명

파일 ①

파일 ②

파일 ③

파일 ④

파일 ⑤

50분

✓ fan-out – 동시에 5명

에이전트 A

에이전트 B

에이전트 C

에이전트 D

에이전트 E

파일 ①

파일 ②

파일 ③

파일 ④

파일 ⑤

10분 + 취합

조건: 서로 독립 같은 파일 금지

예: 파일 10개 마이그레이션 · 라이브러리 후보 3개 비교 조사 · 모듈별 문서화. 벽시계는 줄지만 토큰 총량은 그대로라는 것도 기억하세요.

파이프라인 — 라인은 쉬지 않는다.

과제: 문서 여러 건을 추출 → 번역 → 검수로 처리하기 (단계마다 담당 에이전트 1명)

❌ 배리어 - 단계 전체가 끝나야 다음 **9틱**



✅ 파이프라인 - 끝난 것부터 바로 다음 **5틱**



1번 문서가 검수받는 동안 3번 문서는 추출 중 — 배리어를 없애면 라인이 쉬지 않습니다.

생산 라인에서 이미 익숙한 그림이죠. 항목이 많고 단계가 고정일 때 진가가 나옵니다.

N번 독립 시도, 그리고 채점



🧐 시도들은 서로의 답을 못 봅니다 — 독립성이 다양성을 만듭니다.

💡 적대적 검증: A에게 B를 반박시켜 보세요 — 스스로 매기는 점수보다 훨씬 냉정합니다.

언제? 답 공간이 넓고(설계·문안·전략) 품질이 승부일 때. 비용이 $\times N$ 이 된다는 건 각오하시고요.

조율판 — 상태는 상자 밖에 둡니다

작업 목록(TODO) — 현황판

- [x] 1. API 명세 조사 — 완료 (보고서: *notes/api.md*)
- [>] 2. 파서 구현 — 에이전트 B 진행 중
- [] 3. 테스트 작성 — 2번 완료 후
- [] 4. 문서 갱신

누가 어디까지 했는지가 압축·재시작에도 살아남습니다.

계획서·메모 파일 — 공용 게시판

PLAN.md에 설계와 결정을, NOTES.md에 중간 발견을 — 자식들이 같은 파일을 읽으며 합을 맞춥니다.

자식 A의 발견을 자식 B에게 전하는 가장 싼 방법은, 부모의 입이 아니라 파일입니다.

컨텍스트 세미나의 그 문장 — "상자는 휘발성, 파일은 영속" — 팀에서는 더더욱 진리입니다.

충돌은 공간 분리, 실수는 검문소로

git worktree — 평행 우주 작업 폴더

```
$ git worktree add ../feat-login feature/login  
$ git worktree add ../feat-search feature/search  
# 같은 저장소, 서로 다른 작업 폴더
```

에이전트마다 자기 사본 — 같은 파일을 동시에 고쳐도 **충돌이 없습니다**. 끝나면 브랜치 병합으로 합칩니다.

검증 게이트 — 단계 사이의 검문소

파이프라인 사이사이에 검사 단계를 심습니다 — 테스트 러너 · 린터 · 리뷰 에이전트. 통과 기준은 작업 목록에 명시.

믿지 말고 검사하세요 — 팀원이 로봇일 때는 더욱.

병렬의 두 가지 사고 — 같은 파일 충돌과 불량 결과 전파 — 를 이 두 장치가 막습니다.

오늘 바로 쓰는 위임 문장 4개

 "먼저 서브에이전트로 코드베이스를 조사해 보고서만 받아줘. 그걸로 계획을 세운 뒤 구현해줘."

역할 분담

 "이 파일 10개, 서브에이전트들로 나눠 병렬로 변환해줘. 서로 같은 파일은 건드리지 않게."

fan-out

 "문서마다 추출→번역→검수 순서로, 끝난 것부터 다음 단계로 넘겨줘."

파이프라인

 "서로 다른 접근으로 3번 독립 시도한 뒤, 비교·채점해서 최선을 제안해줘."

심판 패널

코딩을 몰라도 됩니다 — 조직도를 말로 그려주면, 실행은 에이전트가 합니다.

터미널 앞에 없어도 — headless

```
$ claude -p "어젯밤 실패한 테스트를 분석해서  
원인 요약을 report.md로 저장해줘"  
# 대화창 없이 한 번 실행하고 종료 - 결과는 파일과 표준 출력으로
```



스크립트의 부품이 됩니다

반복문·크론 작업에 끼워 넣으면 에이전트가 파이프라인의 한 단계가 됩니다. 매일 밤 로그 분석, 매주 문서 갱신.



에이전트가 에이전트를 부릅니다

오케스트레이터 스크립트가 headless 에이전트 여럿을 지휘 — 사람은 시작과 결과 확인만.

대화형은 "함께 일하기", headless는 "맡겨 두기" — 팀의 야간 근무조입니다.

내가 자는 동안 팀이 일한다

PR 자동 리뷰

PR이 열리면 리뷰 에이전트가 코멘트를 달아줍니다. 사람 리뷰어는 굵은 판단만.

이슈 자동 분류

새 이슈에 라벨·우선순위·담당 제안을 자동으로 — 아침 분류 회의가 짧아집니다.

야간 배치

스케줄 트리거로 매일 밤 — 의존성 점검 · 문서 갱신 · 릴리스 노트 초안.

GitHub Actions 같은 CI에 에이전트를 심으면, **트리거(PR·이슈·스케줄)**가 **상사 역할**을 대신합니다.

아침에 출근하면 리뷰가 달려 있습니다 — 단, 검증 게이트는 여기서도 필수입니다.

팀이 망하는 4가지 방식



과분할

10분짜리 일을 다섯 조각으로 — 위임 오버헤드가 이득을 잡아먹습니다. 회의가 일보다 길어지는 조직처럼요



취합 비용

보고서 5장을 읽고 꺾매는 것도 일입니다 — 반환 형식 통일이 생명 (그래서 위임장에 형식을!)



같은 파일 충돌

자식 둘이 한 파일을 고치면 한쪽 작업이 사라집니다 — 분담 경계를 파일 단위로, 겹치면 worktree



토큰 배수

다중 에이전트는 일반 채팅의 약 15배 토큰을 씁니다 (Anthropic 실측) — 벽시계는 줄고 청구서는 늘니다

네 함정의 공통 처방 — *쫓개기 전에, 쫓갠 가치가 있는지부터.*

판단 플로차트 — 팀이나, 혼자나

작고, 순차적이고, 앞뒤 맥락이 얹혀 있다?

-예→

 혼자 시키세요 — 일상 작업 대부분

아니오 ↓

독립 조각으로 깔끔하게 쪼개지나?

-예→

 fan-out ·  항목×단계면 파이프라인

아니오 ↓

읽을 것은 많은데 결론은 짧은가?

-예→

 역할 분담 — 탐색만 격리해도 절반은 성공

아니오 ↓

답이 여럿이고 품질이 승부인가?

-예→

 심판 패널 (비용 ×N 각오)

아니오 ↓

 혼자 + 검증 게이트 — 그리고 그걸로 충분합니다

헛갈리면 혼자가 기본값입니다 — 팀은 병목이 눈에 보일 때만 부르세요.

 한 장 정리 - 찍어 가세요

패턴 4가지 × 언제 쓰나

패턴	이럴 때	한 줄 요령
 역할 분담	읽을 건 많고 결론은 짧을 때 · 단계가 뚜렷할 때	단계 사이 인수인계는 보고서 파일로
 병렬 fan-out	독립 작업 여러 개 — 파일 10개 변환, 다각도 조사	같은 파일 금지 — 겹치면 worktree로 격리
 파이프라인	항목 많고 단계 고정 — 문서 50건 처리	끝난 항목부터 흘려보내기 — 배리어 제거
 심판 패널	답 공간 넓고 품질 승부 — 설계·문안·전략	독립 시도 → 교차 반박 → 채점 · 비용 ×N
 그 외 대부분	작고 순차적이고 맥락이 얹힌 일	그냥 혼자 + 검증 게이트 — 그게 최적입니다

위임장에는 항상 —  목표 ·  완료 기준 ·  반환 형식

기억할 것, 세 문장

1. 위임의 단위는 컨텍스트입니다 — 자식의 삽질은 자식의 상자에서 끝나게 하세요.
2. 위임장에는 목표 · 완료 기준 · 반환 형식 — 돌아오는 건 요약뿐이라는 전제로 쓰세요.
3. 팀은 공짜가 아닙니다 — 작고 순차적이고 맥락이 얹힌 일은 그냥 혼자 시키세요.

오케스트레이션은 코딩 기술이 아니라 조직 설계입니다 —
그리고 여러분은 이미 조직에서 일해봤습니다.

← [선수 세미나: 컨텍스트](#) · [원본 세미나: 바이브 코딩이란?](#)



이 조직도를 기억하세요 —
질문은 지금, 또는 Q&A 위젯으로 받겠습니다.

REFERENCES

더 깊이 보기

공식 문서 · 도구

[Subagents](#)

Claude Code Docs — 커스텀 서브에이전트 정의·도구 권한

[claude-code-action](#)

GitHub Actions에서 에이전트 실행 — PR 리뷰·이슈 처리

[git worktree](#)

병렬 작업 폴더 — 파일 충돌 격리의 표준 도구

엔지니어링 블로그

[How we built our multi-agent research system](#)

Anthropic — 이 발표의 뼈대 · fan-out과 토큰 15배의 출처

[Claude Code best practices](#)

Anthropic — 워크플로 · headless · worktree 병렬 작업

[Effective context engineering](#)

Anthropic — 컨텍스트 격리·노트 테이킹의 이론적 근거

이 시리즈의 세미나

[컨텍스트 — AI 에이전트의 작업 책상](#)

선수 과목 — 상자의 원리와 격리의 근거

[프롬프트 엔지니어링](#)

위임장 쓰는 법의 원전 — 3요소 프레임

[바이브 코딩이란 무엇인가?](#)

원본 세미나 — 시리즈의 출발점